

Три основных типа задач

Регрессия — предсказать числовое значение y

Классификация — отнести объект к классу

Кластеризация — найти группы похожих объектов

*Также задачи можно разделить на **обучение с учителем** и **обучение без учителя** (ну и просто упомяну про обучение с подкреплением).*

Какие типы какими могут быть?

1. Регрессия:

Предсказываем число

Суть задачи

- **Цель:** Предсказать **непрерывную числовую величину** ($y \in \mathbb{R}$).
- **Вопрос:** «Сколько?»
- **Примеры:**
 - Цена квартиры/алмаза/акции.
 - Температура завтра.
 - Время доставки заказа.
 - Выручка магазина в следующем месяце.

Особенность

Предсказание модели измеряется в тех же единицах, что и целевая переменная (доллары, градусы, минуты).

Обработка данных для регрессии

Данные редко приходят в идеальном виде. Вот основные шаги:

1. Работа с категориальными признаками

Модели понимают только числа. Что делать с колонками вроде **Цвет** или **Город**?

Основой являются два метода

- **One-Hot Encoding** — создает отдельный столбец (0/1) для каждой категории.
- **Ordinal (Label) Encoding** — присваивает категориям числа по порядку (0, 1, 2...).

Когда использовать каждый из методов?

- Когда категории **не имеют порядка** (Цвет: Красный, Синий).
- Когда есть **естественный порядок** (Размер: S, M, L, XL) или категорий очень много.

2. Масштабирование (Scaling)

Зачем?

Если один признак в диапазоне 0-1 (например, рейтинг), а другой 1000-100000 (зарплата), модель может посчитать второй важнее просто из-за масштаба чисел.

Как?:

- Руками через ожидание и дисперсию.
- `StandardScaler` из `sklearn.preprocessing` (имеет интерфейс модели МО).

*Обязательно для **линейных моделей и методов, основанных на расстояниях**. Для деревьев и бустингов — не критично, но полезно для сравнения.*

Параметры масштабирования и кодирования нужно считать **только на обучающей выборке!** Иначе может произойти утечка данных (Data Leakage).

2. Масштабирование (Scaling)

Зачем?

Если один признак в диапазоне 0-1 (например, рейтинг), а другой 1000-100000 (зарплата), модель может посчитать второй важнее просто из-за масштаба чисел.

Как?:

- Руками через ожидание и дисперсию.
- `StandardScaler` из `sklearn.preprocessing` (имеет интерфейс модели МО).

*Обязательно для **линейных моделей и методов, основанных на расстояниях**. Для деревьев и бустингов — не критично, но полезно для сравнения.*

Параметры масштабирования и кодирования нужно считать **только на обучающей выборке!** Иначе может произойти утечка данных (Data Leakage).

```
In [ ]: scaler = StandardScaler().set_output(transform="pandas")
X = scaler.fit_transform(X)
```

Какие модели использовать?

1. Линейные модели (Linear Models)

- `SGDRegressor`, `LinearRegression`.
 - Строят прямую (или гиперплоскость), минимизируя ошибку.
- ✓ Быстрые, интерпретируемые (понятно влияние каждого признака).
- ✗ Плохо работают со сложными нелинейными зависимостями. Требуют масштабирования.

Если зависимость с таргетом при анализе видится как линейная

Какие модели использовать?

1. Линейные модели (Linear Models)

- `SGDRegressor`, `LinearRegression`.
 - Строят прямую (или гиперплоскость), минимизируя ошибку.
- ✓ Быстрые, интерпретируемые (понятно влияние каждого признака).
✗ Плохо работают со сложными нелинейными зависимостями. Требуют масштабирования.

Если зависимость с таргетом при анализе видится как линейная

```
In [ ]: from sklearn.linear_model import SGDRegressor, LinearRegression
```

2. Деревянные модели (Tree Ensembles)

- RandomForestRegressor, GradientBoostingRegressor.
 - Строит множество деревьев решений и усредняет их предсказания.
- ✓ Ловят сложные нелинейные связи, устойчивы к выбросам, не требуют масштабирования.
- ✗ Медленнее, сложнее интерпретировать (нужно разбираться в делениях).

Требуют хорошей настройки параметров для лучшей работы

2. Деревянные модели (Tree Ensembles)

- RandomForestRegressor, GradientBoostingRegressor.
 - Строит множество деревьев решений и усредняет их предсказания.
- ✓ Ловят сложные нелинейные связи, устойчивы к выбросам, не требуют масштабирования.
- ✗ Медленнее, сложнее интерпретировать (нужно разбираться в делениях).

Требуют хорошей настройки параметров для лучшей работы

```
In [ ]: from sklearn.ensemble import RandomForestRegressor
```

3. CatBoost / XGBoost / LightGBM

- Продвинутые градиентные бустинги.
 - Часто дают лучшее качество (SOTA), но не всегда.
- ✓ Умеют работать с категориями «из коробки» (CatBoost).
- ✗ Интерпретировать еще сложнее.

Самое то для бэйзлайнов и быстрой проверки гипотез

3. CatBoost / XGBoost / LightGBM

- Продвинутые градиентные бустинги.
 - Часто дают лучшее качество (SOTA), но не всегда.
- ✓ Умеют работать с категориями «из коробки» (CatBoost).
✗ Интерпретировать еще сложнее.

Самое то для бэйзлайнов и быстрой проверки гипотез

```
In [ ]: from catboost import CatBoostRegressor
```

Как понять, хорошо ли у нас получилось?

MAE (Mean Absolute Error)

$$\frac{1}{n} \sum \|y - \hat{y}\|$$

*Буквально средняя ошибка **в единицах измерения таргета**.*

✓ Легко интерпретировать.

✗ Не сильно «штрафует» за сильные отклонения.

Как понять, хорошо ли у нас получилось?

MAE (Mean Absolute Error)

$$\frac{1}{n} \sum \|y - \hat{y}\|$$

*Буквально средняя ошибка **в единицах измерения таргета**.*

✓ Легко интерпретировать.

✗ Не сильно «штрафует» за сильные отклонения.

```
In [ ]: from sklearn.metrics import mean_absolute_error
```

RMSE (Root Mean Squared Error)

$$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

Корень из средней квадратичной ошибки. Можно и без корня, но так масштабы понятнее.

✓ Нормально так штрафует за сильные отклонения.

✗ Менее интерпретируема (**квадраты единиц измерения** и корень из них).

RMSE (Root Mean Squared Error)

$$\sqrt{\frac{1}{n} \sum (y - \hat{y})^2}$$

Корень из средней квадратичной ошибки. Можно и без корня, но так масштабы понятнее.

✓ Нормально так штрафует за сильные отклонения.

✗ Менее интерпретируема (**квадраты единиц измерения** и корень из них).

```
In [ ]: from sklearn.metrics import root_mean_squared_error, mean_squared_error
```

R^2 (R-squared)

$$1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

*Доля объясненной дисперсии (от $-\infty$ до 1). Показывает, насколько **модель лучше простого среднего**.*

✗ Может расти при добавлении лишних признаков.

R^2 (R-squared)

$$1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Доля объясненной дисперсии (от $-\infty$ до 1). Показывает, насколько **модель лучше простого среднего**.

✖ Может расти при добавлении лишних признаков.

```
In [ ]: from sklearn.metrics import r2_score
```

R^2 (R-squared)

$$1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

Доля объясненной дисперсии (от $-\infty$ до 1). Показывает, насколько **модель лучше простого среднего**.

✗ Может расти при добавлении лишних признаков.

```
In [ ]: from sklearn.metrics import r2_score
```

Смотрите на связку **MAE** (для понимания средней ошибки) и **RMSE** (чтобы увидеть наличие катастрофических промахов). R^2 используйте для общего сравнения моделей.

Почему RMSE и MAE могут быть неудобны?

- Они зависят от масштаба данных (доллары, килограммы, проценты).
- Нельзя сравнить качество моделей на разных масштабах.

MASE (Mean Absolute Scaled Error)

$$MASE = \frac{MAE_{model}}{MAE_{naive}}$$

Показывает ошибку модели **относительно ошибки наивного прогноза**.

- **MASE < 1**: модель лучше, чем просто брать наивное значение.
- **MASE = 1**: модель работает так же, как наивный прогноз.
- **MASE > 1**: модель хуже наивного прогноза.

В отличие от R^2 легко интерпретируема, так как работает с абсолютной ошибкой.

MAPE (Mean Absolute Percentage Error)

$$MAPE = \frac{100\%}{n} \sum \left| \frac{y - \hat{y}}{y} \right|$$

*Показывает **на сколько процентов** мы ошиблись относительно реального значения.*

- ✓ Легко **интерпретируется**, не используя единицы измерения.
- ✓ Можно сравнивать качество прогноза для разных категорий/данных.
- ✗ Если есть нули, то метрика сломается.

Регрессия: итог

1. **Задача:** Предсказать число.

2. **Данные:**

- Категории → **One-Hot** (нет порядка) или **Ordina** (есть порядок).
- Числа → `StandardScaler` (особенно для линейных моделей).

3. **Модели:**

- Быстро и просто → Линейная регрессия.
- Дольше, но с качеством → `Random Forest` / `CatBoost`.

4. **Метрики:** *MAE*, *RMSE*, R^2 .

5. **Важно:** Данные делить на train/test надо до обработки.

2. Классификация:

Предсказываем класс

Суть задачи

- **Цель:** Отнести объект к одному из **классов** ($y \in \{0, 1, \dots, K\}$).
- **Вопрос:** «Куда это?» или «Да/Нет?».
- **Примеры:**
 - Спам или не спам? (*Бинарная*)
 - Уйдет клиент или останется? (*Бинарная*)
 - Это кошка, собака или птица? (*Многоклассовая*)
 - Диагностика заболевания (??).

Особенность

Часто встречается **дисбаланс классов** (одних объектов много, других мало). Например, мошеннических транзакций всего 1%, а обычных 99%. Это меняет подход к оценке качества.

Обработка данных для классификации

1. Кодирование признаков

Аналогично регрессии:

- **One-Hot Encoding** для номинальных признаков.
- **Ordinal Encoding** для порядковых.
- Целевую переменную (y) тоже нужно превратить в числа (например, "Yes" → 1, "No" → 0).

Обработка данных для классификации

1. Кодирование признаков

Аналогично регрессии:

- **One-Hot Encoding** для номинальных признаков.
- **Ordinal Encoding** для порядковых.
- Целевую переменную (y) тоже нужно превратить в числа (например, "Yes" → 1, "No" → 0).

```
In [ ]: from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder  
        from sklearn.compose import ColumnTransformer  
        from sklearn.pipeline import Pipeline
```


2. Масштабирование

- Нужно для **логистической регрессии** и методов, использующих градиентный спуск или расстояния (`KNN` , `SVM`).
- Для деревьев (`RandomForest` , `CatBoost`) — опционально.

3. Борьба с дисбалансом

- **Методы семплирования:** Undersampling (удалить часть большинства), Oversampling (создать синтетические примеры меньшинства).
- **Настройка моделей:** Параметр `class_weight='balanced'` во многих моделях sklearn автоматически увеличивает штраф за ошибку на редком классе.

Многоклассовая Классификация (Multi-class Classification)

Что делать, если классов больше двух?

1. Обработка целевой переменной

- **Label Encoding:** Присваивает числа 0, 1, 2...
 - ✓ *Подходит* для деревьев и ансамблей, которые не чувствительны к порядку чисел.
 - ✗ *Не подходит* для линейных моделей, если они воспринимают числа как порядок (хотя обычно используют One-Hot для таргета внутри функции потерь).

2. Стратегии обучения (если модель бинарная по природе)

Некоторые алгоритмы (например, логистическая регрессия в базе) умеют только различать «Да/Нет». Как быть?

- **One-vs-Rest (OvR):** Строим несколько моделей. Каждая модель учится отличать *свой* класс от *всех остальных*, выбираем класс с наибольшей вероятностью.
- **One-vs-One (OvO):** Строим модель для каждой пары классов. Голосование определяет победителя.

Классические модели для классификации

1. Логистическая регрессия (Logistic Regression)

- `LogisticRegression`
- Оценивает вероятность принадлежности к классу через сигмоиду.

✓ Быстрая, выдает вероятности.

✗ Линейная граница решения.

Хороша для бэйзлайна

Классические модели для классификации

1. Логистическая регрессия (Logistic Regression)

- `LogisticRegression`
 - Оценивает вероятность принадлежности к классу через сигмоиду.
- ✓ Быстрая, выдает вероятности.
✗ Линейная граница решения.

Хороша для бэйзлайна

```
In [ ]: from sklearn.linear_model import LogisticRegression
```

2. Деревья решений и ансамбли

- **RandomForestClassifier**
 - ✓ Надежный метод, редко переобучается, легко настраиваются параметры.
- **Gradient Boosting (CatBoost , XGBoost)**
 - ✓ Часто дает наилучшее качество на табличных данных.
 - ✓ CatBoost особенно хорош с категориальными признаками (?), хорошо работает из коробки.

3. Метод опорных векторов (SVM)

- ✓ Хорошо работает в задачах с большим количеством признаков.
- ✗ Медленен на больших выборках.

Какие метрики использовать?

Accuracy

*Самая очевидная метрика, которая показывает **долю верно предсказанных ответов**.*

Но нам не всегда достаточно только точности (например при дисбалансе классов). Поэтому нужно грамотно подбирать метрики.

Confusion Matrix (Матрица ошибок)

	Predicted 0	Predicted 1
Actual 0	TN (Верно отриц.)	FP (Ложная тревога)
Actual 1	FN (Пропуск цели)	TP (Верно полож.)

Простые, но хорошие метрики основаны именно на матрице ошибок.

Precision (Точность)

$$\frac{TP}{TP + FP}$$

Среди тех, кого модель назвала "1", сколько реально "1"?

► Если ложные срабатывания дороги (например, блокировка хорошего клиента).

Precision (Точность)

$$\frac{TP}{TP + FP}$$

Среди тех, кого модель назвала "1", сколько реально "1"?

► Если ложные срабатывания дороги (например, блокировка хорошего клиента).

```
In [ ]: from sklearn.metrics import precision_score
```

Recall (Полнота)

$$\frac{TP}{TP + FN}$$

Среди всех реальных "1", скольких модель нашла?

► Если пропуск цели критичен (поиск больных, мошенников, терактов).

Recall (Полнота)

$$\frac{TP}{TP + FN}$$

Среди всех реальных "1", скольких модель нашла?

► Если пропуск цели критичен (поиск больных, мошенников, терактов).

```
In [ ]: from sklearn.metrics import recall_score
```

F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Гармоническое среднее Precision и Recall. Баланс между точностью и полнотой. Обычное среднее арифметическое было бы слишком «добрым» к экстремальным значениям.

Модель имеет `precision = 1.0` и `recall = 0.0`.

Арифметическое среднее: $(1 + 0)/2 = 0.5$ (выглядит неплохо).

F1-score: 0 (так как один из множителей равен нулю, вся метрика обнуляется).

F1-Score

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Гармоническое среднее Precision и Recall. Баланс между точностью и полнотой. Обычное среднее арифметическое было бы слишком «добрым» к экстремальным значениям.

*Модель имеет `precision = 1.0` и `recall = 0.0`.
Арифметическое среднее: $(1 + 0)/2 = 0.5$ (выглядит неплохо).*

F1-score: 0 (так как один из множителей равен нулю, вся метрика обнуляется).

```
In [ ]: from sklearn.metrics import f1_score
```

ROC-AUC

Обычно модель классификации выдает **вероятность** (от 0 до 1), а не сразу класс 0 или 1.

- Стандартный порог: 0.5. Если $\mathbb{P} > 0.5 \rightarrow$ класс 1.
- Но что если мы изменим порог на 0.1? Мы поймем больше мошенников (recall вырастет), но будет много ложных тревог (precision упадет).
- Что если порог 0.9? Мы будем очень точны, но пропустим многих мошенников.

Как оценить модель в целом, не привязываясь к одному порогу?

ROC Curve (Receiver Operating Characteristic)

Это график, который показывает качество модели при **всевозможных порогах** классификации.

- **Ось X:** FPR (False Positive Rate) — доля ложных тревог среди всех негативных объектов.

$$FPR = \frac{FP}{FP + TN}$$

- **Ось Y:** TPR (True Positive Rate) — то же самое, что **recall**.

$$TPR = \frac{TP}{TP + FN}$$

Идеальная кривая: проходит через левый верхний угол (Recall=1, FPR=0).

Случайная кривая: диагональ из (0,0) в (1,1) (модель гадает монеткой).

AUC (Area Under Curve)

Площадь под кривой ROC. Одно число, характеризующее качество модели.

- **AUC = 1.0:** Идеальный классификатор. Разделяет классы без ошибок.
- **AUC = 0.5:** Бесполезный классификатор (работает как случайное угадывание).
- **AUC < 0.5:** Модель работает хуже случайности (возможно, вы перепутали метки классов).
- **AUC = 0.85:** Хороший результат. В 85% случаев модель оценит случайно выбранного положительного объекта выше, чем случайно выбранного отрицательного.

AUC (Area Under Curve)

Площадь под кривой ROC. Одно число, характеризующее качество модели.

- **AUC = 1.0:** Идеальный классификатор. Разделяет классы без ошибок.
- **AUC = 0.5:** Бесполезный классификатор (работает как случайное угадывание).
- **AUC < 0.5:** Модель работает хуже случайности (возможно, вы перепутали метки классов).
- **AUC = 0.85:** Хороший результат. В 85% случаев модель оценит случайно выбранного положительного объекта выше, чем случайно выбранного отрицательного.

```
In [ ]: from sklearn.metrics import roc_auc_score
```

Классификация: итог

1. **Задача:** Предсказать категорию/класс.

2. **Данные:**

- Кодирование категорий + Масштабирование (для линейных моделей).
- Проверка на дисбаланс классов.

3. **Модели:**

- База → Логистическая регрессия.
- Что-то круче → RandomForest / CatBoost.

4. **Метрики:**

- Не только Accuracy.
- Precision, Recall, F1-Score, ROC-AUC с интерпретацией.

5. **Важно:** Выбор метрики в зависит от бизнес-задачи.

3. Кластеризация

Суть задачи

- **Тип обучения: Unsupervised Learning** (без учителя).
- **Цель:** Разбить объекты на группы (**кластеры**) так, чтобы объекты внутри одной группы были похожи друг на друга, а на объекты из других групп — не похожи.
- **Особенность: НЕТ правильных ответов.** Мы не знаем заранее, сколько должно быть групп и кто куда относится.
- **Примеры:**
 - Сегментация клиентов (богатые/бедные, экономные/транжиры).
 - Группировка новостей по темам.
 - Поиск аномалий (объекты, не попавшие ни в один кластер).

Особенность

Результат часто требует интерпретации человеком. Алгоритм скажет «это группа 1», а аналитик должен понять, какие у нее характеристики.

Обработка данных для кластеризации

1. Масштабирование

- Алгоритмы кластеризации (например, K-Means) используют **расстояния** (евклидово) между точками.
- Если один признак имеет диапазон 0-1, а другой 0-10000, то второй признак полностью определит расстояние, и первый будет проигнорирован.

2. Отбор признаков

- Шумные или нерелевантные признаки могут сильно ухудшить качество кластеризации.
- Нужно брать только самые важные признаки, имеющие смысл для группировки.

3. Кодирование

- Категориальные признаки нужно закодировать (One-Hot), но это может сильно увеличить размерность. Иногда проще убрать категории или использовать специальные метрики расстояний.

Классические модели для кластеризации

1. K-Means (Метод K-средних)

1. *Выбираем K центров случайно.*
2. *Приписываем точки к ближайшему центру.*
3. *Пересчитываем центры как среднее точек в кластере.*
4. *Повторяем шаги 2-3 до сходимости.*

- ✓ Очень быстрый, простой, хорошо работает на сферических кластерах.
- ✗ Чувствителен к выбросам. Предполагает, что кластеры выпуклые и одинакового размера.

2. DBSCAN

- Группирует точки, лежащие плотно друг к другу.* ✓ Не нужно задавать число кластеров. Находит кластеры произвольной формы. Выделяет выбросы (шум).
- ✗ Плохо работает, если плотность кластеров сильно различается.

Метрики качества кластеризации

Так как правильных ответов нет, мы используем **внутренние метрики**.

1. Inertia (Инерция) ↓

Сумма квадратов расстояний от точек до центров их кластеров. ✗ Инерция всегда уменьшается с ростом числа кластеров K . Нельзя просто взять минимум. Используют «Метод локтя» (ищем излом на графике).

2. Silhouette Score (Коэффициент силуэта) ↑

Оценивает, насколько точка похожа на свой кластер по сравнению с другими кластерами. ✓ Помогает выбрать оптимальное K : выбираем то, где Silhouette Score максимален.

Лучше не просто верить чиселкам, а смотреть на графики.

Кластеризация: итог

1. **Задача:** Найти скрытые группы без размеченных данных.

2. **Данные:**

- **StandardScaler — строго обязателен** (иначе признаки с большими числами доминируют).
- Удаление шума и выбросов желательно.

3. **Модели:**

- Стандарт → K-Means (быстро, просто).
- Сложные формы/выбросы → DBSCAN.

4. **Метрики:**

- Inertia (метод локтя).
- Silhouette Score (чем выше, тем лучше).

5. **Важно:** Выбор числа кластеров K — это подбор + метрики. Результат нужно интерпретировать предметно.

Важные нюансы

1. Утечка данных через масштабирование (Data Leakage)

- ⊗ Сделать `scaler.fit(X)` на всем датасете до разделения на Train/Test.
- ✓ Использовать `Pipeline`, чтобы это происходило автоматически.

2. Проклятие размерности (Curse of Dimensionality)

При использовании **One-Hot Encoding** для признаков с сотнями категорий матрица данных становится гигантской и разреженной.

- ✓ Объединять редкие категории в одну группу `"0ther"`.
- ✓ Использовать модели, любящие категории, которые не требуют One-Hot.

3. Интерпретация кластеров

- ✓ После обучения K-Means сделайте `.groupby('Cluster')` и посмотрите на разные статистики.

Прежде чем обрабатывать данные, ответьте на вопрос: «Для какой задачи и модели я это делаю?»

Золотые правила предобработки

Данные никогда не бывают идеальными. Чтобы модель работала честно:

- **Кодирование:** Превращаем текст в числа.
 - Нет порядка? → `One-Hot Encoding`.
 - Есть порядок или много категорий? → `Ordinal Encoding` / `Target Encoding`.
- **Масштабирование (`StandardScaler`):**
 - **Обязательно** для: Линейных моделей, K-Means, KNN, SVM (алгоритмы, считающие расстояния или градиенты).
 - **Не критично** для: Деревьев решений и ансамблей (Random Forest, CatBoost), но иногда полезно.
- **Разделение данных:**
 - Сначала `train_test_split`, потом `fit` преобразователей.
 - Нарушение этого правила ведет к **Data Leakage** (утечке данных) и завышенной оценке качества.

